

Neural Network Classification of

Alzheimer's disease from MRI images

Presentado por:

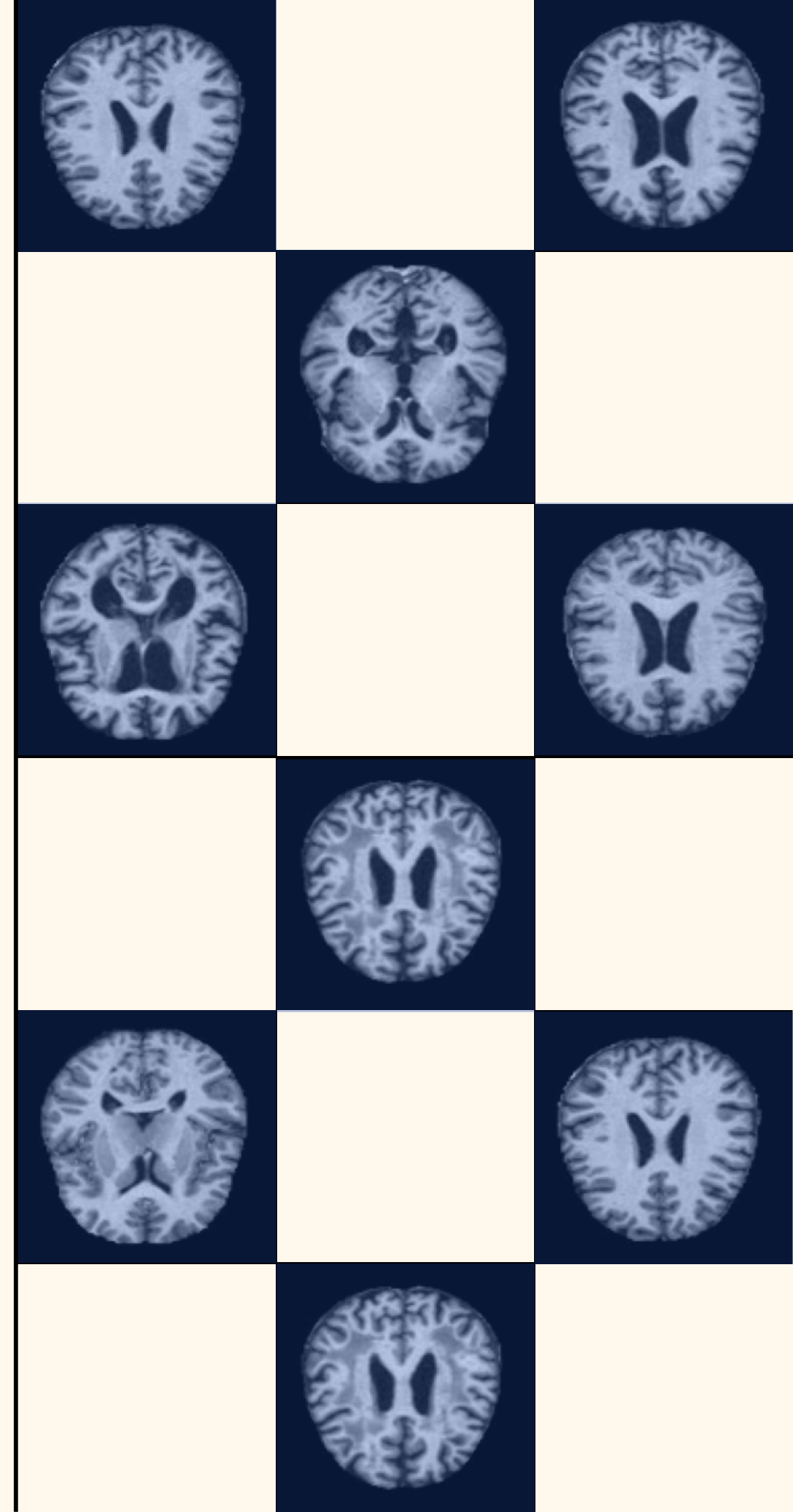
Valeria Ontaneda

Tutor:

Yunier Córdova Cobas

Presented Date:

07.30.2026



Alzheimer's disease

Progressive neurodegenerative disorder that gradually destroys memory, thinking skills and the ability to perform every day tasks.

Caused by a build up of proteins in the form of amyloid plaques (beta-amyloid) and neurofibrillary tangles (tau protein) in the brain. This causes neurons to stop communicating and die overtime, shrinking the brain.

Once diagnosed, it does not have a cure but early detection plays a big role in slowing down progression and improving quality of life.

Clinical Dementia Rating Stages (CDR)

Used for labeling as it categorizes patients based on their Alzheimer progression

➔	Normal	0	Normal aging
➔	Very Mild	0.5	Slight memory impairment, fully independent in self-care.
➔	Mild	1	Moderate loss of recent memory, daily activities impairment.
➔	Moderate	2	Significant memory loss & disorientation. Requires personal care assistance.
➔	Severe	3	Severe cognitive loss, inability to recognize people. Loss of motor skills.

**Can we use MRI images to
detect Alzheimer's disease
using Neural Networks?**

Process Walkthrough

01

**Cleaning
Datasets**

02

**Preprocessing
images**

03

**Splitting dataset
for training and
testing**

04

Model training

05

Final evaluation

Tools used

Google Colab

Kaggle

Pandas
Numpy

Scikit-learn

Seaborn
Matplotlib

Nibabel



Keras: The Interface. Designs the network layer by layer.

TensorFlow: The Engine. Makes heavy calculations of tensors and mathematics

Python: The Base. The language where the logic is written

Cleaning Datasets

Open access series of Imaging Studies (OASIS)

Neuroimaging datasets of the brain made freely available to study Alzheimer's disease.

Database 1

198 patients

3D processed files

Removed patients with no CDR scores

Filtered age ≥ 60 as below that there is only non-demented cases.

+

Database 2

120 patients

3D RAW files

All patients had CDR scores

Filtered first visit MRIs to keep everything at a patient level - 1 visit (MRI session)

=

Final Database

318 patients

2D images separated between non-demented and alzheimers

CDR scores 0, 0.5, 1 and 2

MRI session per patient
Age ≥ 60

CDR Score

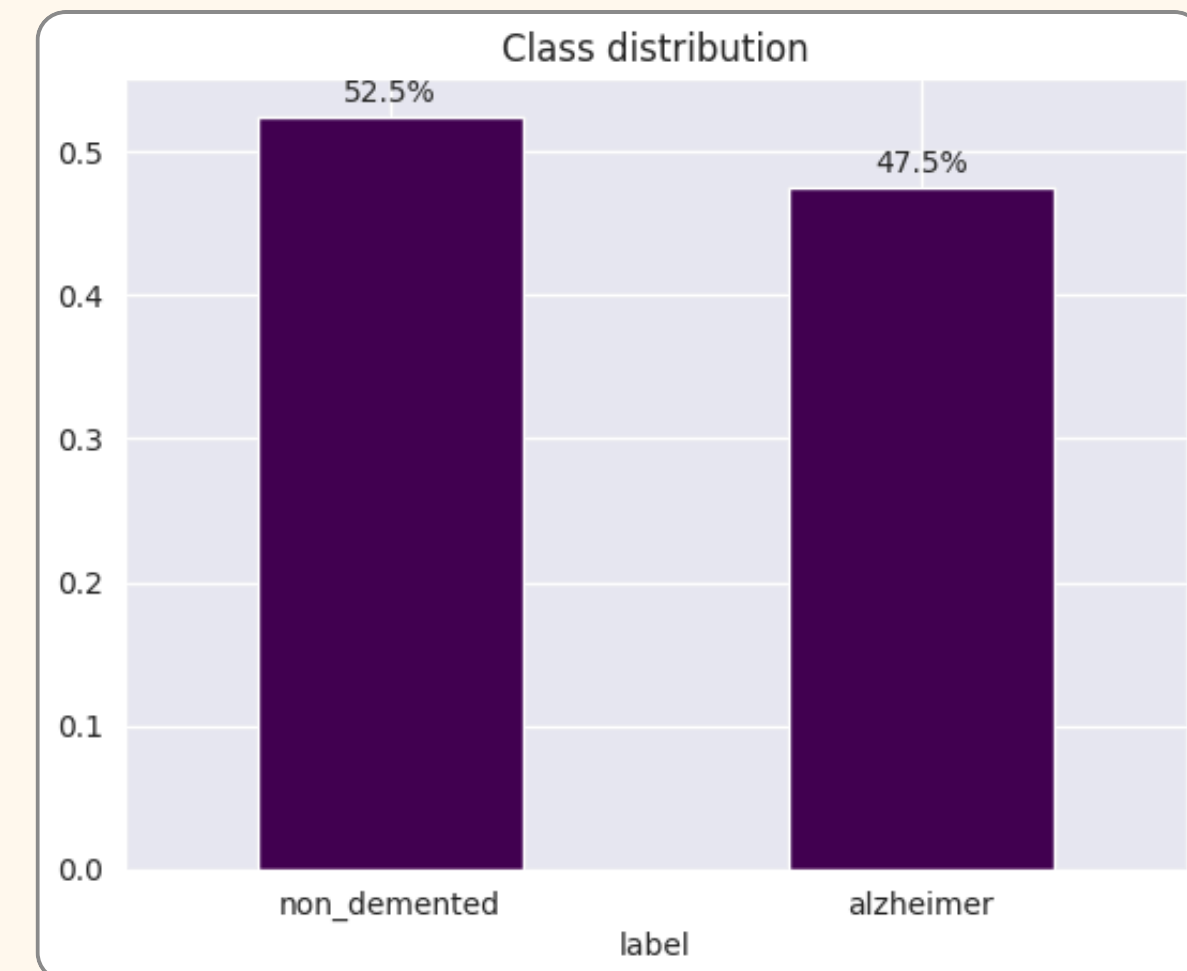
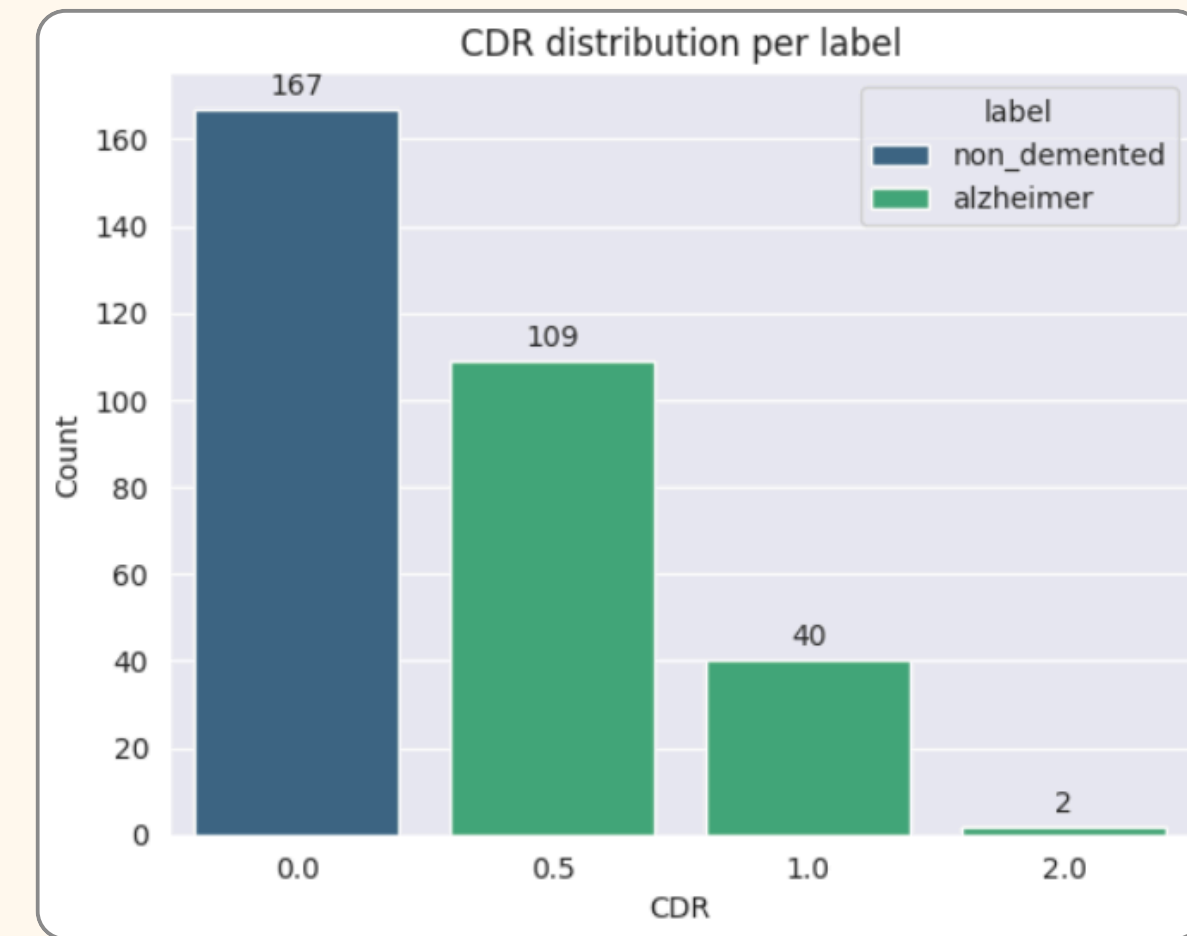
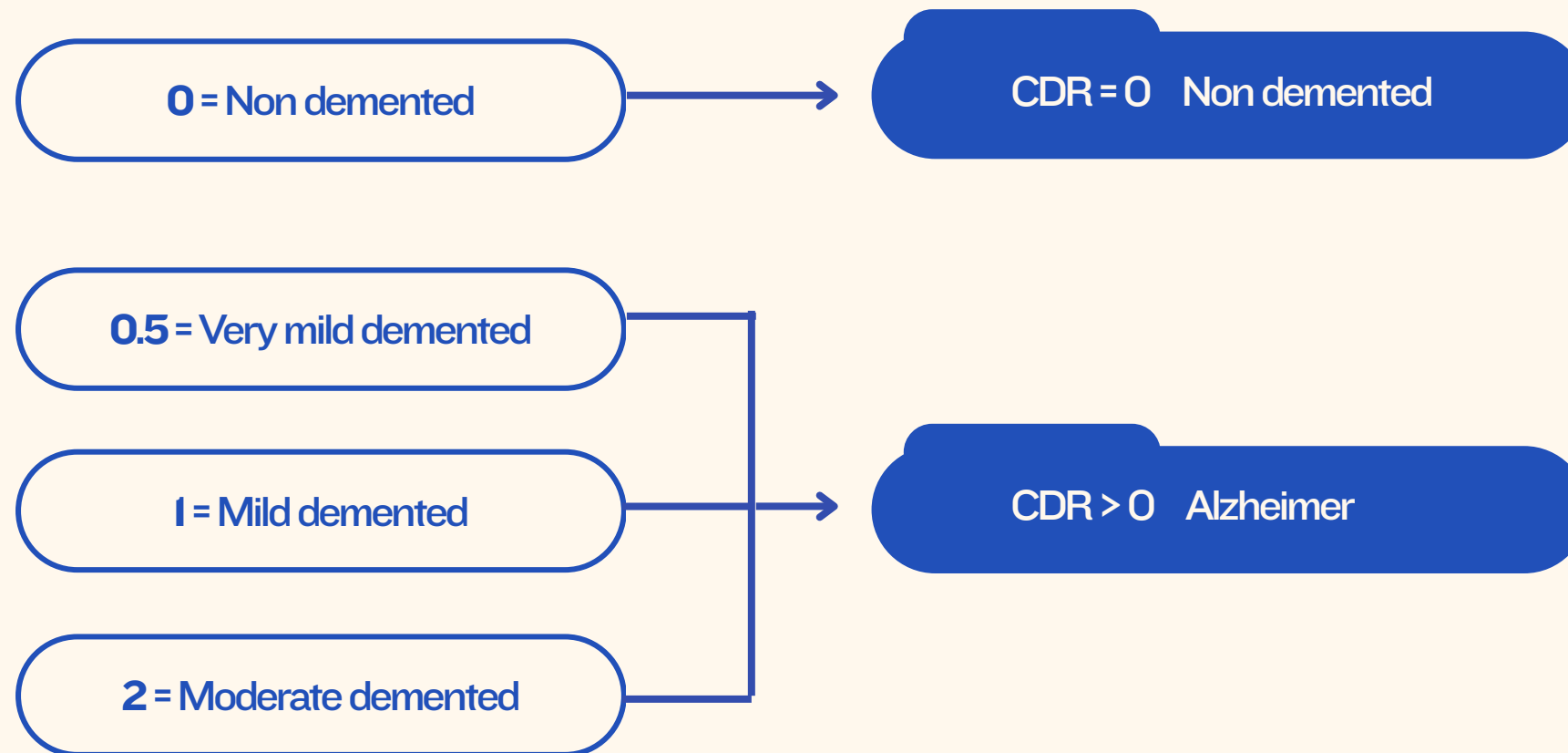


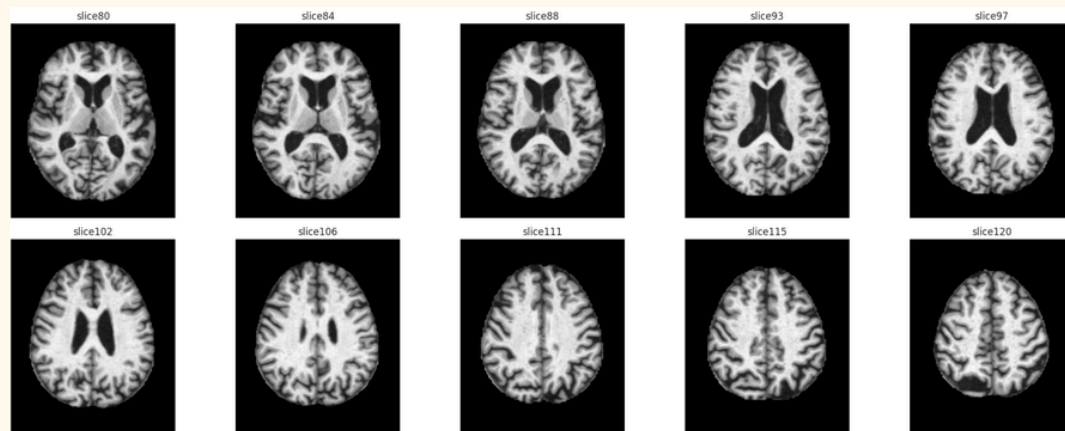
Image Preprocessing

Database 1

Processed 3D files

Extracted slices from axial/transverse view

From the middle of the brain to ensure covering parts that have evidence of Alzheimer's disease.



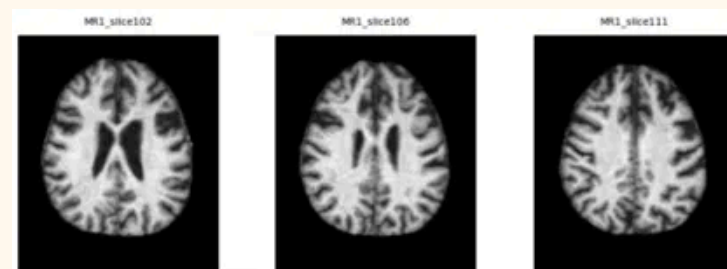
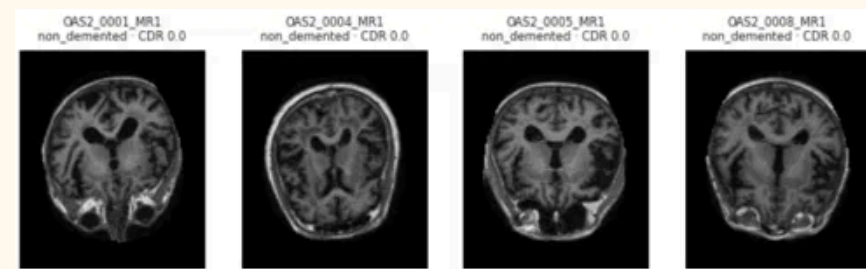
Database 2

RAW files

Removed skull and face

Corrected brightness

Corrected positioning



Output: 3180 images

10 PNG images of different slices of the patient's brain of the axial plane

Slices from the midsection to view lateral ventricles and sulci for easier Alzheimer detection

Stripped of skull and face to reduce noise

Similar brightness between datasets

Dataset Split

70%

Training

222 patients

15%

Validation

48 patients

15%

Testing

48 patients

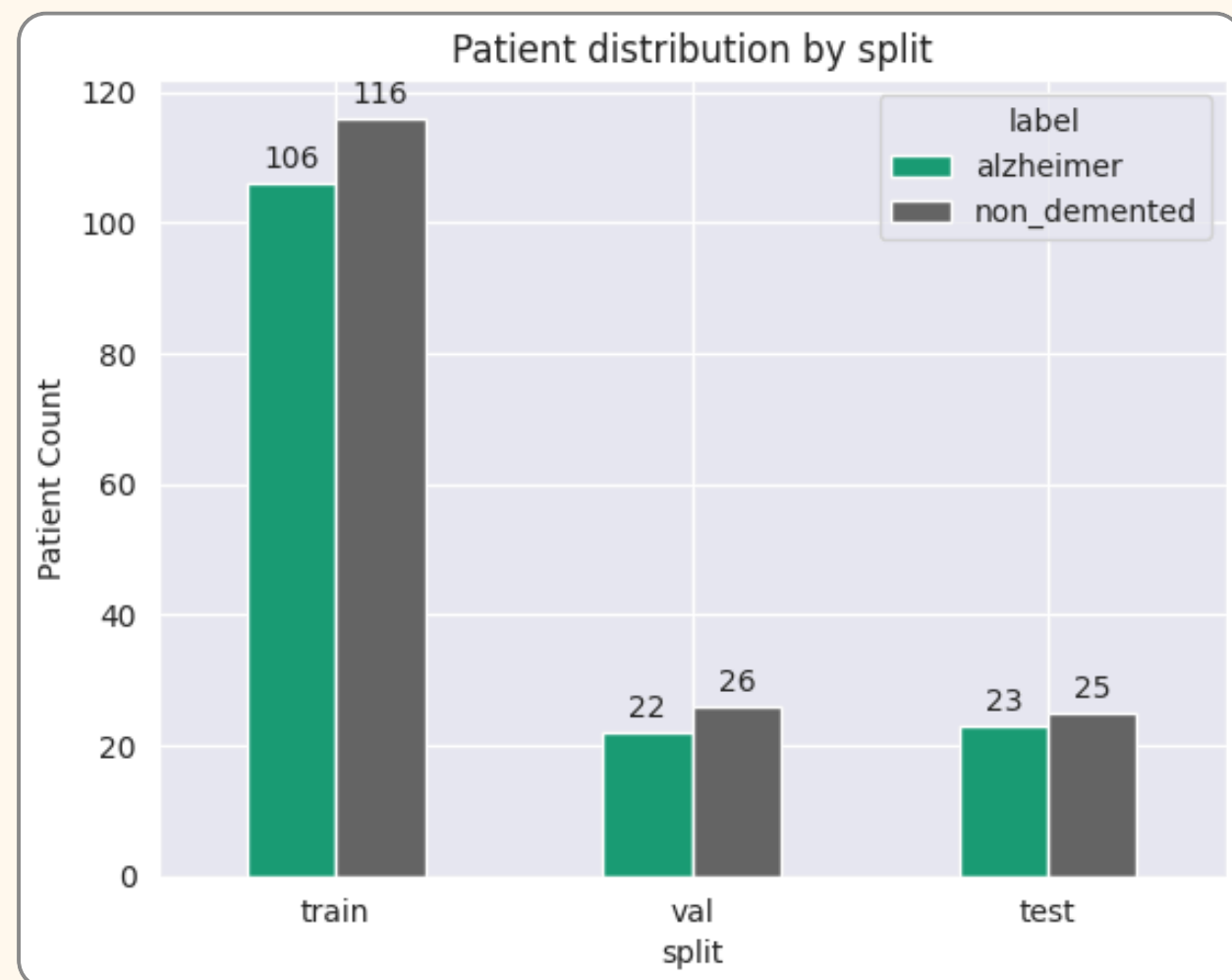
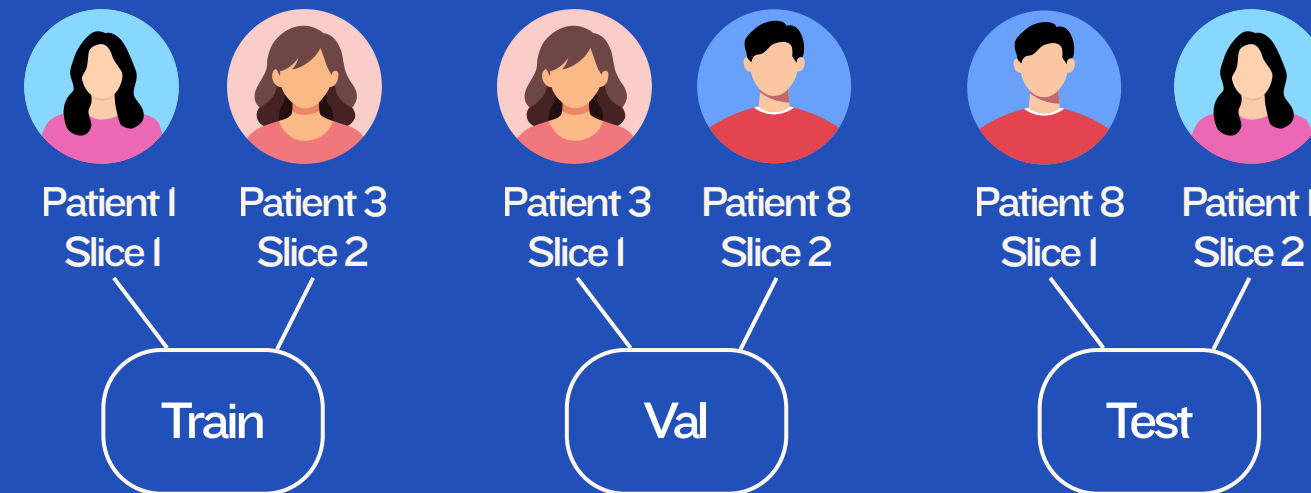
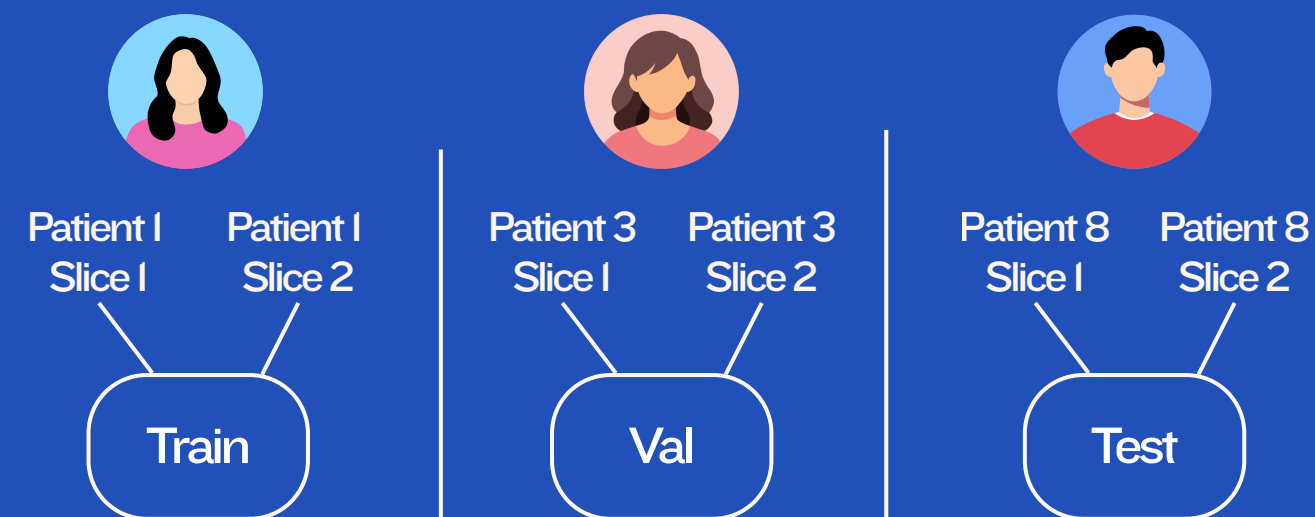


Image split



You are training your model with the exam answers

Patient split



Keeps datasets separate and test data intact

Model Plan

01

**Baseline:
Dummy Classification**

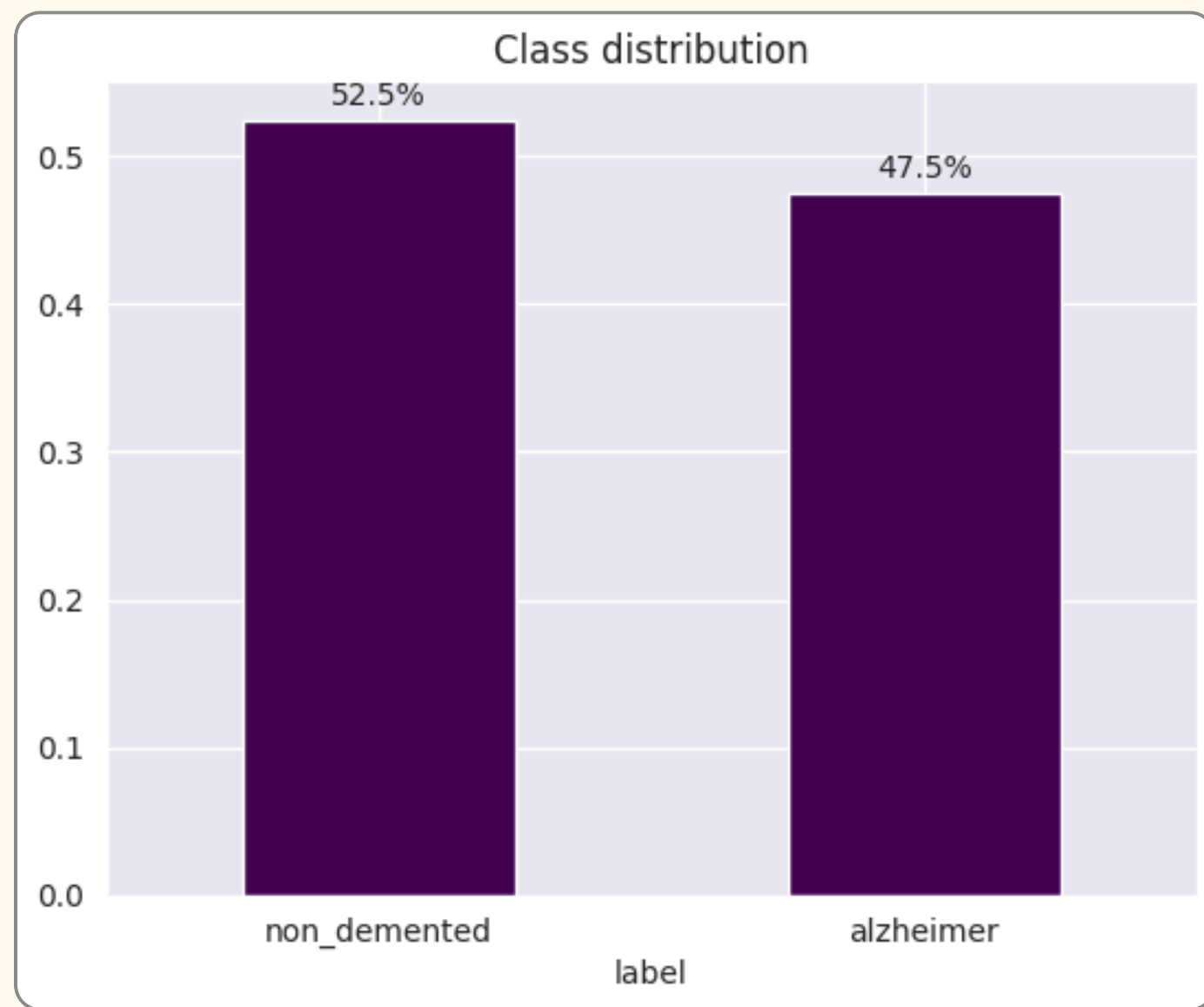
02

**CNN:
From scratch**

03

**Transfer Learning:
EfficientNetB0**

Dummy Classifier



Accuracy: 54%

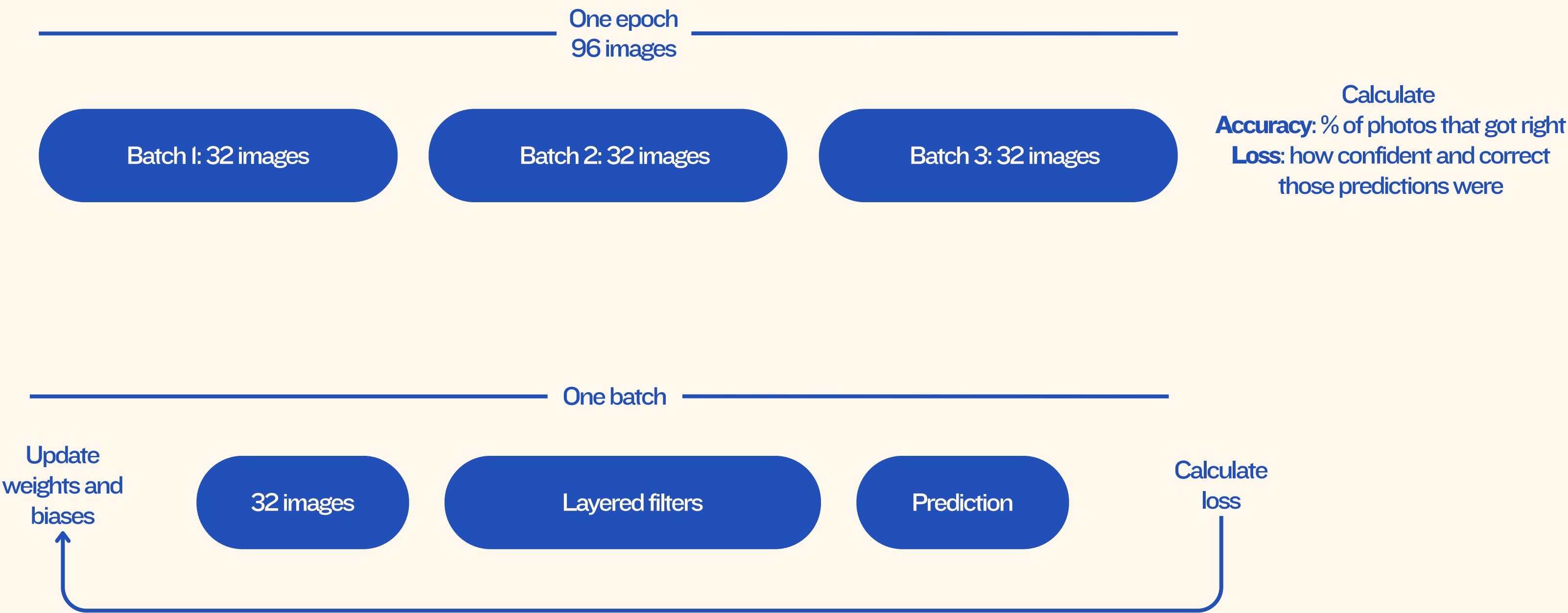
The scikit-learn DummyClassifier model serves as a baseline.

It determines the majority class, in this case **non_demented** and predicts non_demented all the time.

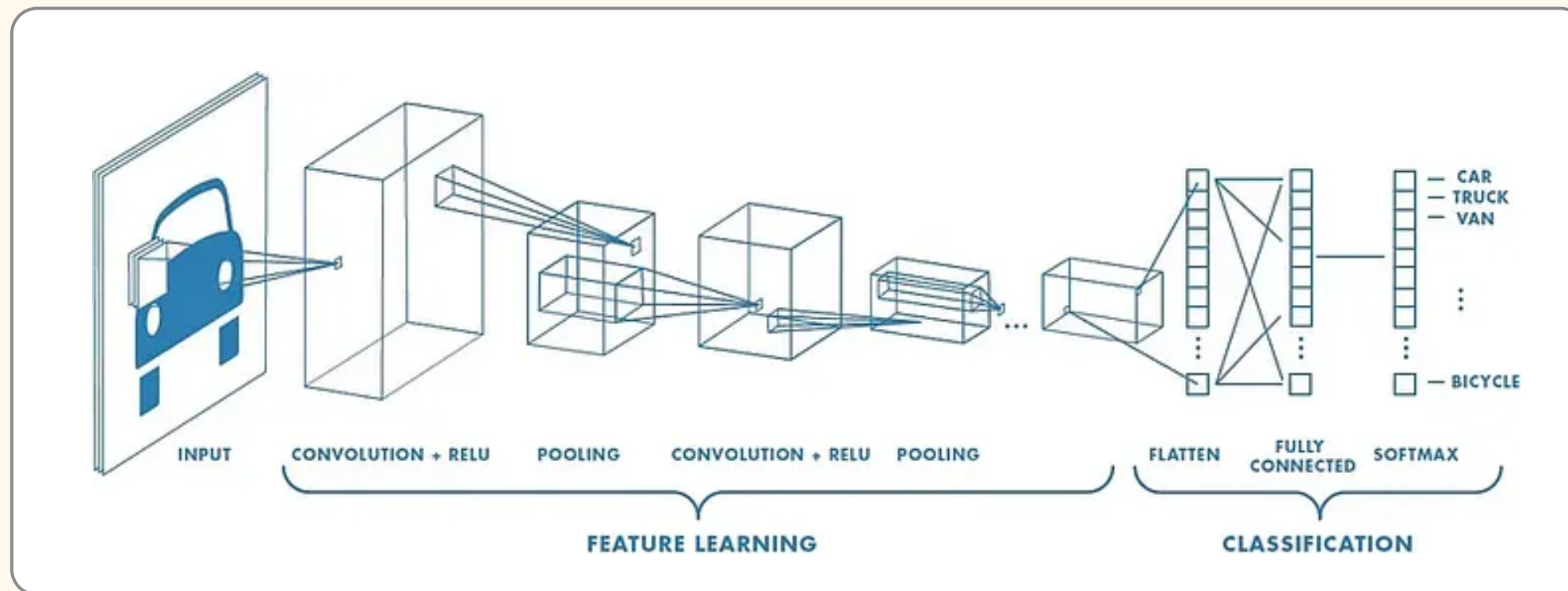
The goal is that the next models need to beat the 54% of accuracy.

CNN overview

Convolutional Neural Network (CNN): A specialized deep learning architecture designed to process visual data by using layered filters to automatically detect patterns, features, and objects in images.



CNN overview



Feature Learning

- **Input:** Raw image data (pixels) fed into the network for processing.
- **Convolution:** Applies learnable filters across the image to detect visual patterns like edges, textures, and shapes.
- **ReLU (Activation):** Replaces negative values with zero, introducing non-linearity so the network can learn complex, real-world patterns.
- **Pooling:** Downsamples the feature maps to reduce spatial size, cut computation costs, and make pattern detection invariant to small shifts or movements.

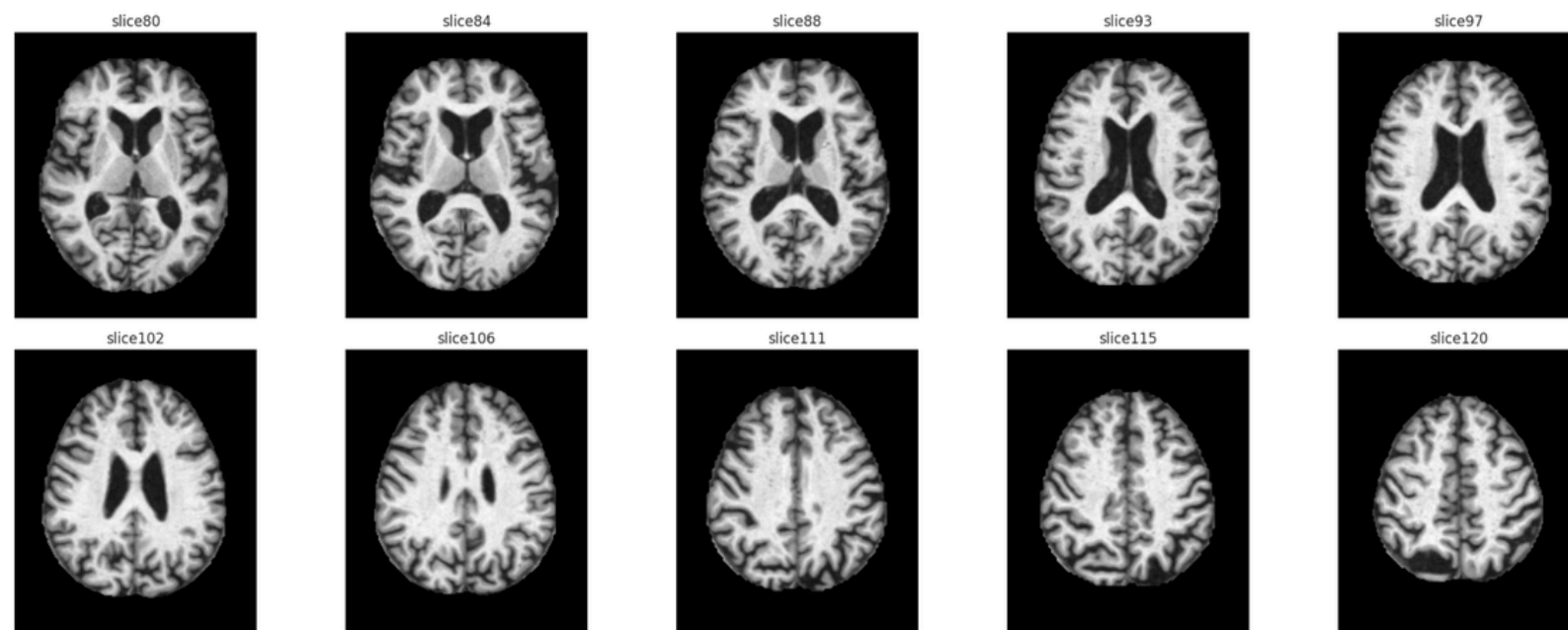
Classification

- **Flatten:** Converts the 3D feature blocks into a single 1D vector (a long list of numbers) so traditional dense layers can read it.
- **Fully Connected:** Combines all extracted features together to "reason" which specific combination of patterns corresponds to each class.
- **Softmax (Activation):** Converts the final layer's raw output scores into a set of probability percentages that sum to 100% (e.g., Non_demented: 92%).

Patient level evaluation



A single patient

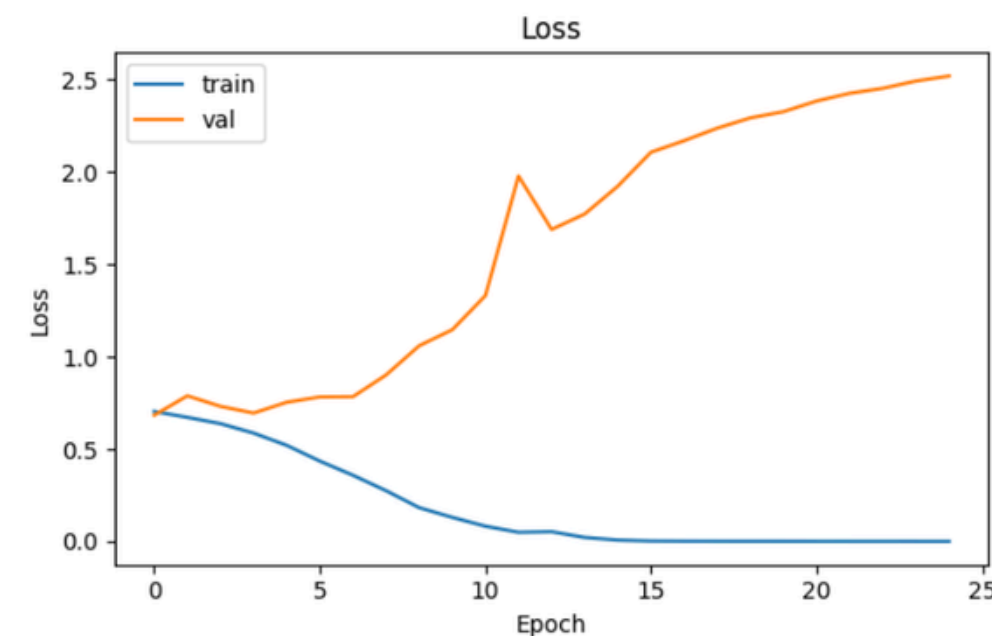
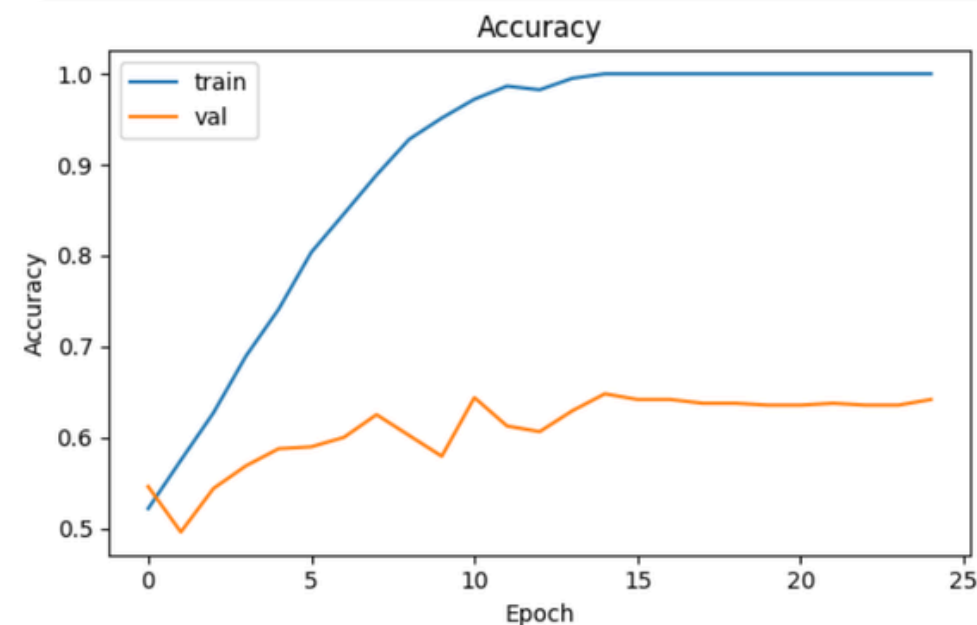


Prediction average
of the 10 slices

CNN from scratch

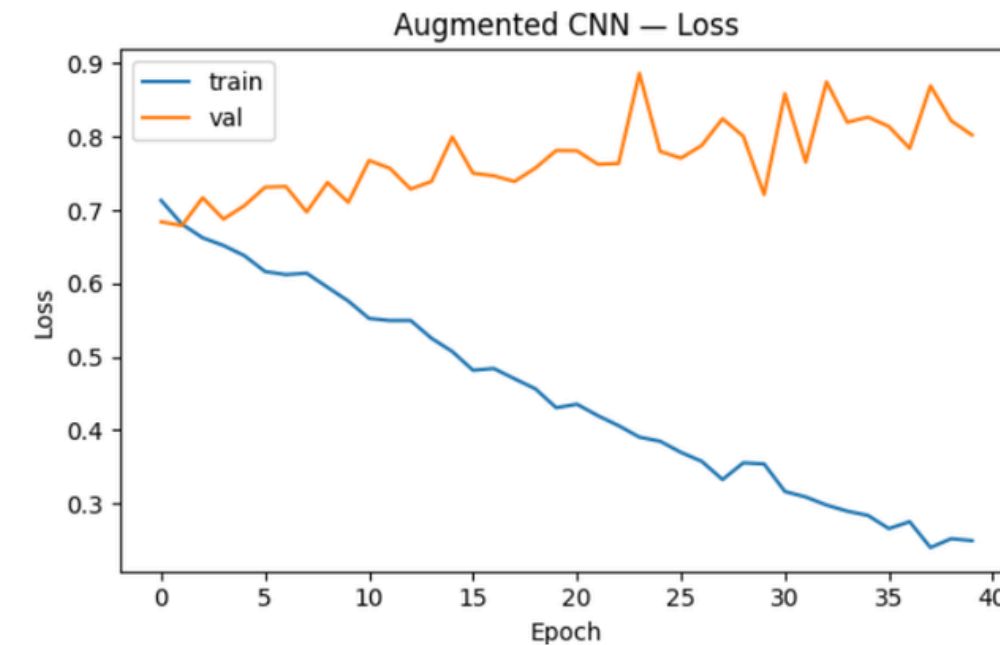
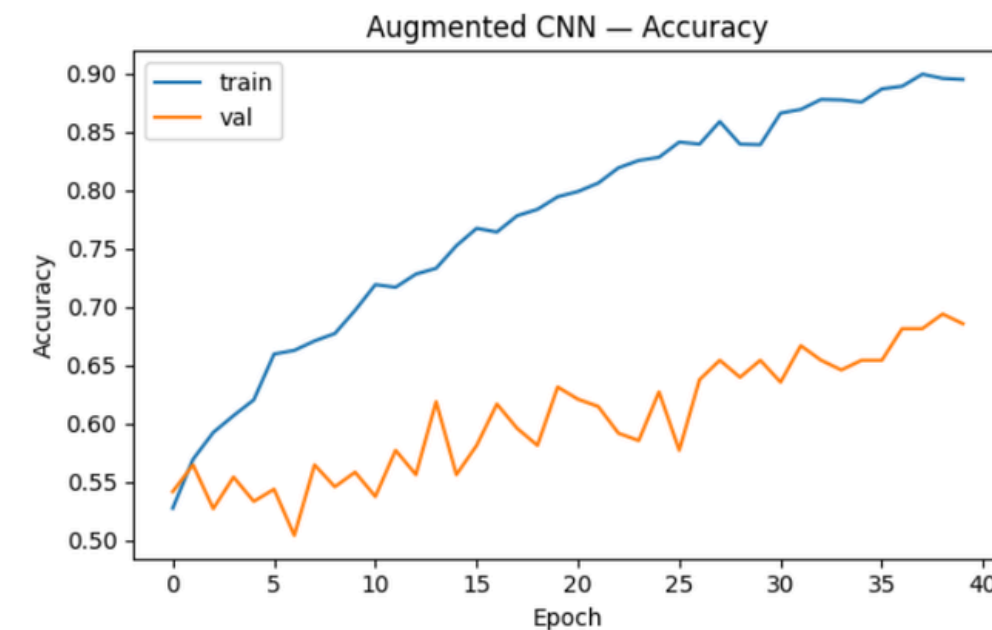
Basic 3-block CNN, of convolutional layer combined with Max Pooling

Without Augmentation:



Accuracy: 71%

With Augmentation and Early Stopping:



Accuracy: 77%

Overfitting:

Train accuracy is high but val accuracy is low and the difference grows bigger per epoch
The model is memorizing images instead of understanding general patterns

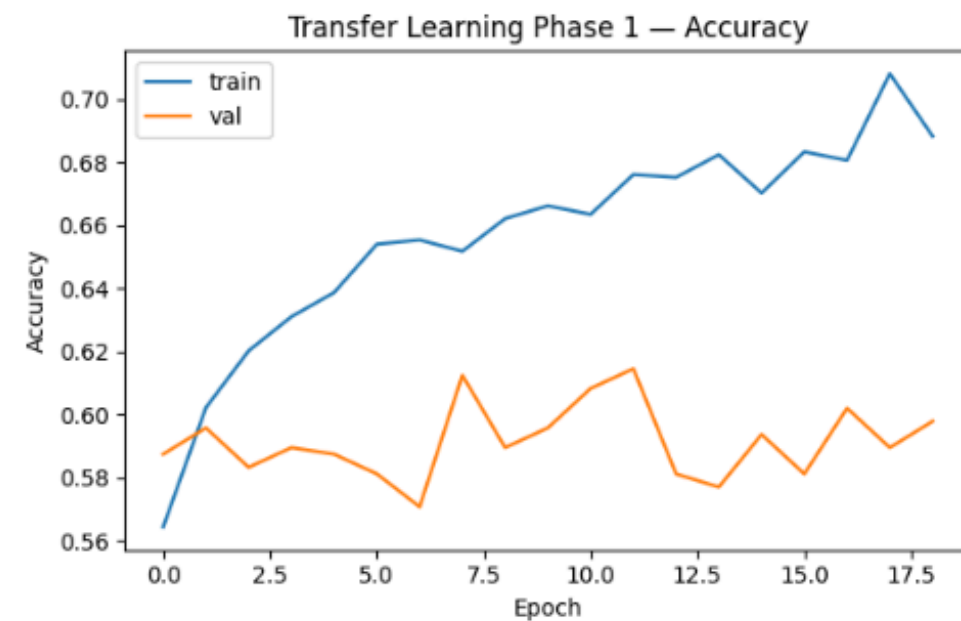
Augmentation: During training, new versions of the images are created with slight modifications, which help with overfitting as the model does not memorize an image.
Rotation and translation were used.

Early stopping: Based on the val_accuracy it stops, you plug in the patience so let's say 7 and if it keeps decreasing after 7 epochs it automatically stops

Transfer Learning

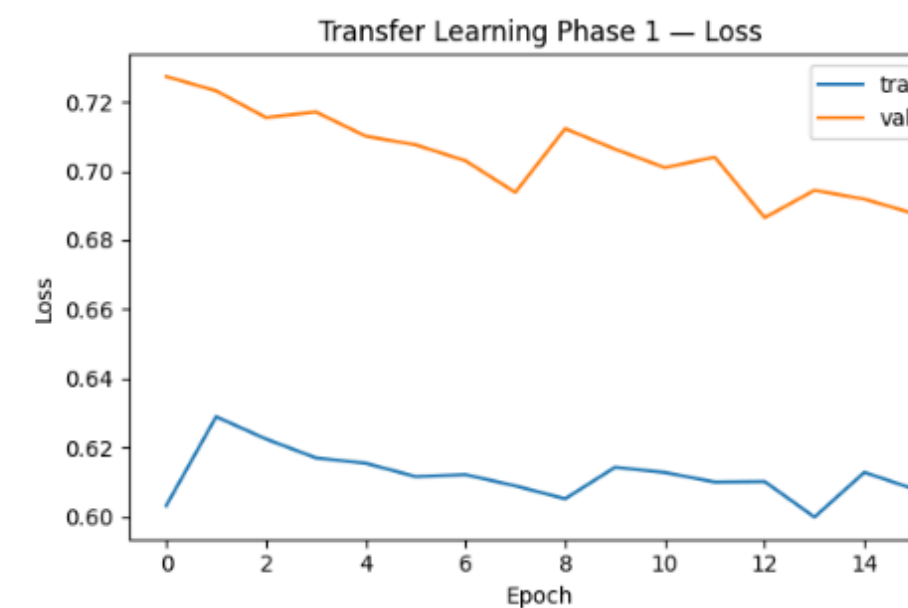
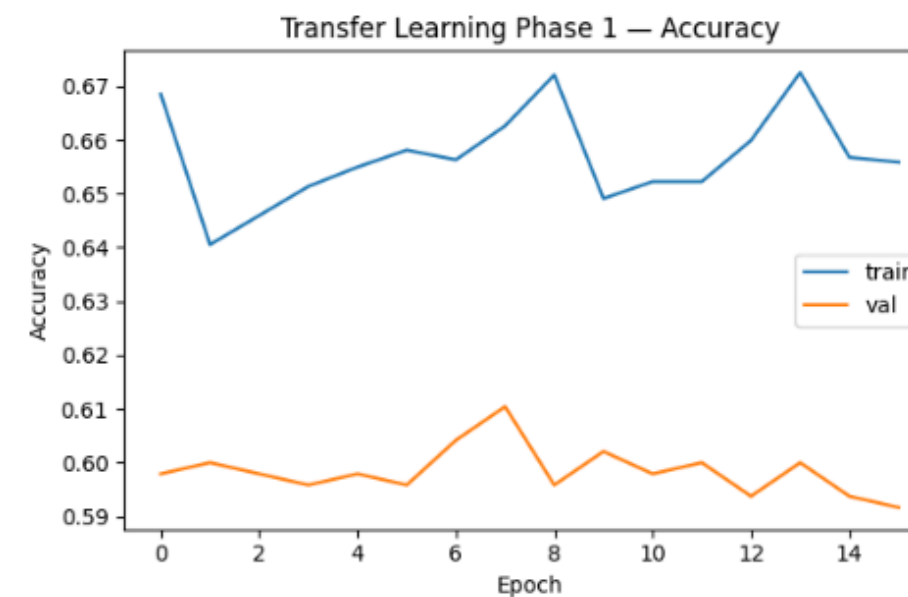
EfficientNetB0 is a network pretrained on 1.4 million photos, adapted in two phases to recognize brain atrophy

Phase 1:



Accuracy: 71%

Phase 2:



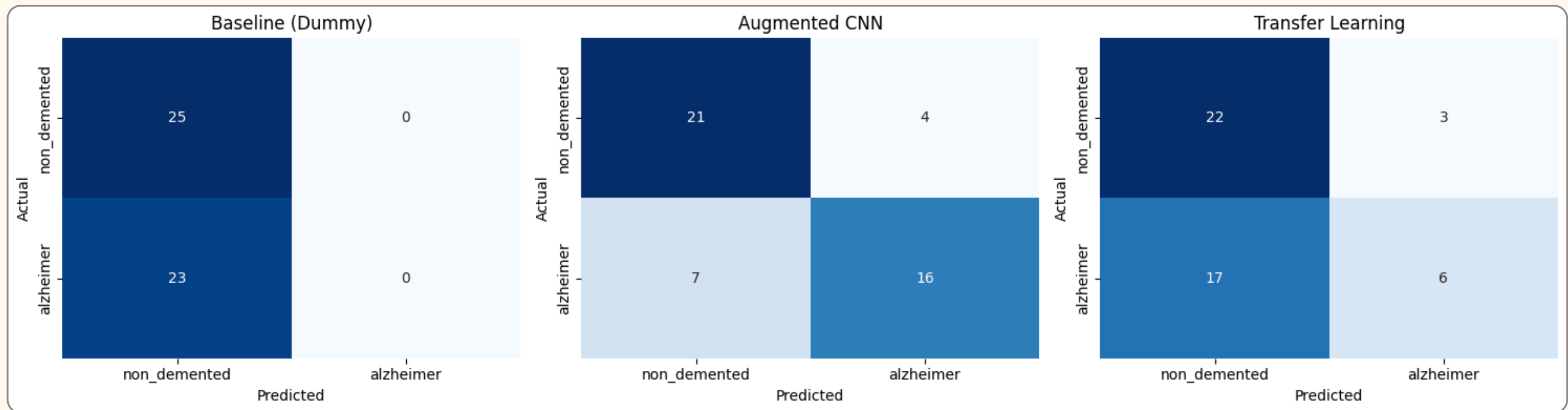
Accuracy: 77%

Phase 1: Freeze the entire pretrained network. Only train a small new classification head (~164,000 new parameters, versus millions in the frozen part). This alone got us to about 61.5% validation accuracy.

Phase 2 (fine-tuning): Unfreeze just the last 20 layers of the pretrained network, and re-train with a much smaller learning rate — nudging the pretrained features slightly toward "brain MRI" instead of "everyday photos," without wrecking what they already know.

Test results

Testing models at the end of all training with the dataset from test



Accuracy: 52%
F1 (macro): 34%

Accuracy: 77%
F1 (macro): 77%

Accuracy: 58%
F1 (macro): 53%

Conclusions

Can we use MRI images to detect Alzheimer' disease using Neural Networks?

Yes, it shows promising results. A compact CNN trained from scratch, with light, physiologically appropriate augmentation and early stopping, reached 77% test accuracy, outperforming both a majority-class baseline and an EfficientNetB0 transfer learning model

However, due to a small dataset there is still evidence of overfitting that could be improved

Data leakage produces unrealistic results

Image-level splitting gave 100% accuracy; splitting by patient (the correct method) dropped that to 77%. This is a pattern also seen in many public Kaggle notebooks using these same OASIS datasets

Augmentation must respect the signal

Heavy augmentation destroyed the atrophy pattern the model needs to detect; light augmentation + early stopping helped

Limitations and Future Work

- **Small dataset (318 patients total)** → Main bottleneck throughout the project; combining OASIS-1 + OASIS-2 already helped meaningfully.
 - **CDR class imbalance** → “Alzheimer” groups all CDR > 0 patients but is mostly CDR 0.5 (very mild), where atrophy is often too subtle to see on MRI — weakening the signal.
-
- **RadImageNet pretrained backbone** → Medically-pretrained features could beat ImageNet transfer
 - **3D CNN architectures** → Could use full volumetric information instead of 2D slices
 - **Data leakage quantification** → Test the augmented CNN with an image-level split to measure the leakage effect directly, rather than relying only on patterns from public notebooks

Thank you